Data integrity / Integralność danych
Data models / Modele danych
E-R diagrams / Diagramy E-R

dr hab. inż. Maciej Grzenda
M.Grzenda@mini.pw.edu.pl
http://www.mini.pw.edu.pl/~grzendam

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Warsaw University of Technology

MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Some challenges related to data quality

- How to ensure that the data stored in our database is high quality?
  - How to avoid problems such as multiple records with the same primary key value present in a database? This could happen due to human mistakes or errors in client applications placing data in databases.
  - How to avoid removing, e.g. a customer record to which orders refer? Could we identify who placed the order and where to ship the products then?
  - Is it better to prevent data quality issues or fix them later?

The answer: DBMS capabilities, which (if properly used) will prevent many data quality problems

Data integrity

Data integrity

- Several aspects of integrity:
  - Entity integrity (*integralność encji*),
  - Referential integrity (*spójność referencyjna*),
  - Additional constraints (*dodatkowe ograniczenia*)
- Modern DBMSs, when configured properly, reject data modification statements that violate data integrity e.g. by:
  - Violating the uniqueness of a primary key
  - Violating referential integrity

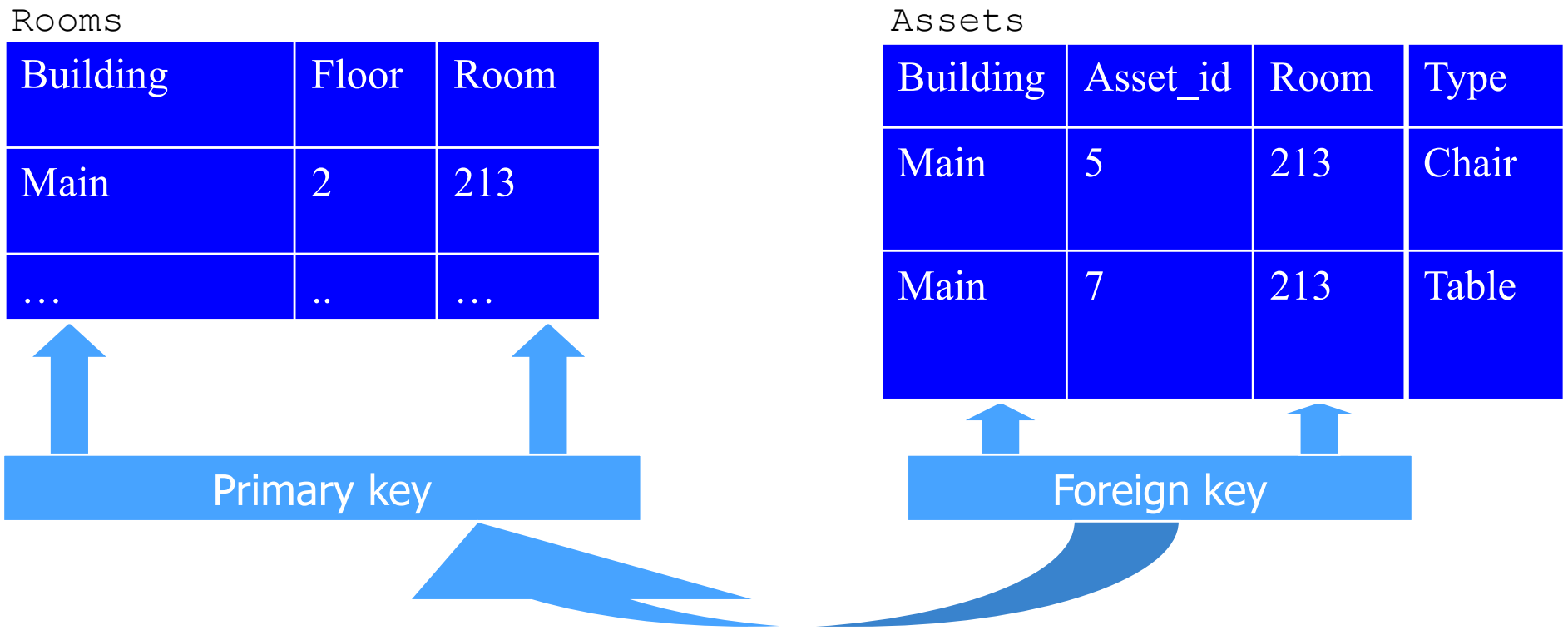
Entity integrity

- Objective: guarantee the uniqueness of a primary key throughout table records
- Solution:
  - primary key is defined for a table by the person defining the table
  - typically, a unique index, i.e. a tree-like structure used to quickly check which PK values already exist in a table, is automatically created by a DBMS as well
  - next, the DBMS monitors all the create and update operations to maintain the uniqueness of a PK. It rejects requests to modify table data that would result in having records with duplicate PK values

Referential integrity

- Objective: guarantee that foreign keys refer to existing records identified by matching PK values
- Solution:
  - a primary key is defined for a primary key table and a foreign key is defined for a foreign key table to establish the relation
  - The DBMS monitors all the CRUD (create/update/delete) operations to maintain the relation

Referential integrity revisited



In the analysed case, DBMS will not allow:

- Removing a `Rooms` record for which an `Assets` record with same (building,room) exists
- Changing PK of a `Rooms` record for which an `Asset` record with same (building,room) exists
- making an `Asset` record refer to non-existing (building,room) in `Rooms` table

Surrogate primary keys
Zastępcze klucze główne

- A frequent practice is to replace natural primary keys with surrogate primary keys
- A surrogate primary key
  - is a PK with integer values generated from a sequence, typically 1,2,....
  - relies on values not used in the real world, frequently not even shown to end users
  - is a solution when no natural PK exists e.g. when enterprise asset management is implemented for the first time
  - is also frequently used in modern databases, especially when a natural PK is a composite PK

Referential integrity – surrogate PKs in use

Rooms

Room_id	Building	Floor	Room
23215	Main	2	213
23216	…	..	…
…	…	…	…

Assets

Asset_id	Room_id	Asset	…
423	**23215**	Chair	…
756	**104576**	Table	…

Referential integrity is absolutely necessary, when a primary key (hence, a foreign key as well) is a surrogate PK. Should room record with room_id=23215 be removed, nothing will be known about the location of asset 423.

Additional constraints

- It is possible to define logical conditions that must be fulfilled by all the records like:
  - salary>tax to ensure that the value in salary column is greater than the value of tax for each person in a table
- These conditions are checked whenever CRUD operations are attempted on a table
- However, such conditions may depend on local configuration settings, needed by individual enterprises. Hence, a tendency to place such constraints in client applications rather than DBMSs is observed.

Some challenges related to understanding real-life needs of the users of database systems

- Can we somehow document our understanding of business needs? Will our understanding always be correct?
- Can we somehow confirm with business experts that indeed we understand the way business works?
  - Will they be able to understand primary keys and foreign keys?
  - If so, is there a way to use less technical language and share our understanding of data needs with non-IT professionals?

The answer is:

data models and entity-relationship diagrams

Data models

This part of the lecture is mostly based on S. Hoberman, ***Data Modeling Made Simple*, 2nd Ed., 2016, Technics Publications**

Data model

"*A data model is a set of symbols and text used for communicating a precise representation of an information landscape*"

Steve Hoberman, Data Modeling Made Simple, Technics Publications 2016

A data model:

- **contains only types**, not actual values such as real names, prices or VAT IDs
- **contains interactions**, i.e. it captures how concepts interact with each other, e.g. whether a company has one or many phone numbers
- **provides a concise communication medium**. A piece of paper with a data model communicates more than multiple spreadsheet files with hundreds of real records of e.g. persons, products, companies etc.

Do we need data models?

" *(…) a good data model serves as a lingua franca between business and information-technology professionals. (…) Conceptual and logical data models capture the ways in which technical professionals think a business process works. Business professionals can examine these assumptions and offer corrections and refinements before code is created*"

Foreword to Steve Hoberman, Data Modeling Made Simple, Technics Publications 2016

The categories of data models

Model format	Idea	Comments
Conceptual (*model pojęciowy, ewent. konceptualny*)	Conceptual data model (CDM) represents the business at a very high level	Contains **only basic** (frequently used by organisation) and **critical** (fundamental for the organisation) **concepts** with their definitions and relationships. **Limited to one page** and **ca. 20 concepts of top importance** e.g. for project scope.
Logical	Logical data model (LDM) represents a detailed business solution	Contains **entities including all attributes**, according to strict business rules and **independent of technology**
Physical	Physical data model (PDM) represents a detailed technology solution, optimised for a specific software or even hardware	Is a **LDM modified with performance-enhancing techniques for a specific environment** e.g. Ms SQL Server

We focus on logical data models in this lecture, as these document the way organisation works in detail, while they refrain from including aspects unique for the database software (such as Oracle Database or MongoDB) to be used. Based on logical models, physical models will be developed, which will be discussed during next lectures.

The use of data models

- Data models are used:
  - during the analysis and design phases of a project before the actual database is created,
  - to understand an existing application,
  - to understand the impact of planned changes to both business and data models,
  - to learn about the business, as a data model helps understand the way the organisation works.
- Which data model should be used depends on the needs, e.g.
  - To explain how a legacy sales application works to a team of data scientists, the model selected should be the physical data model with the following features:
    - Scope: department
    - Time: today (we focus on existing solutions not the ones planned in the future)
    - Function: application

Entity-relationship modelling

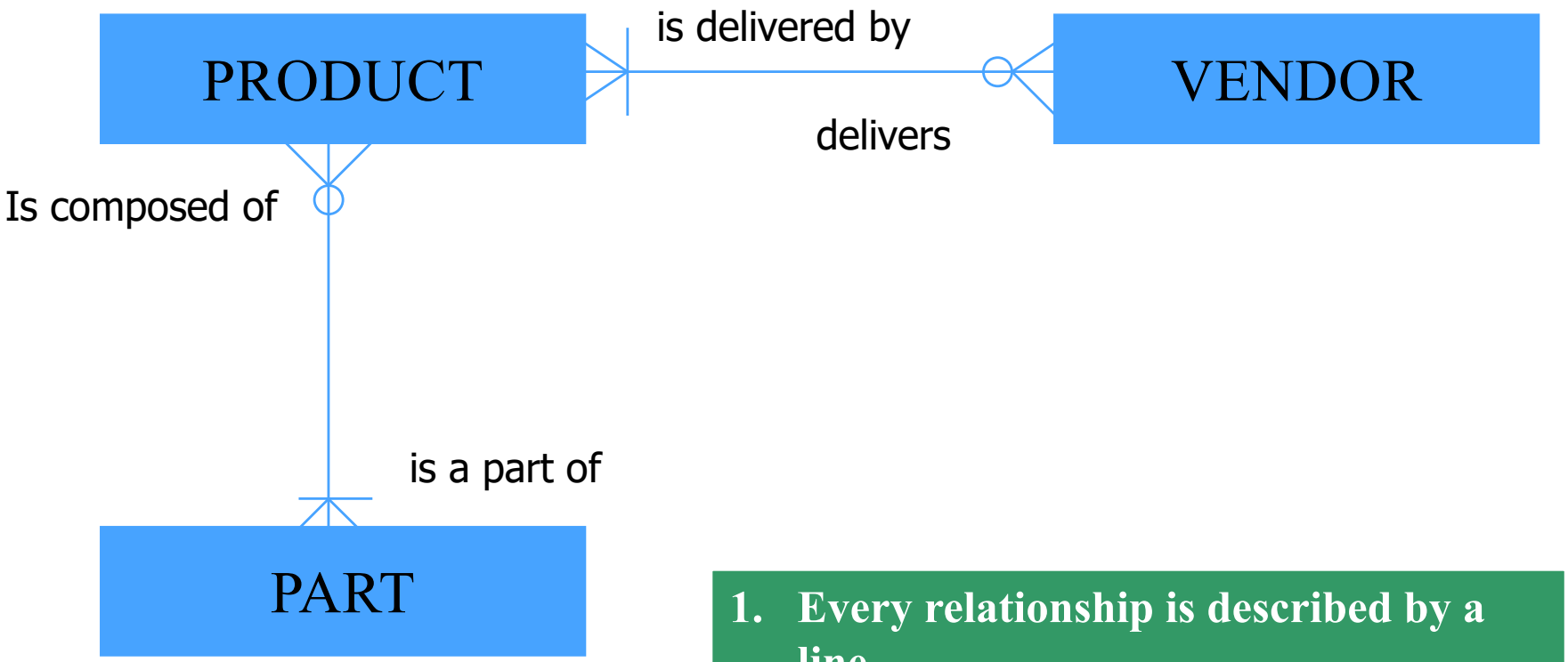
Terminology

- Entity – a real world object – e.g. patient, employee, order, contract, …
- Entity class a.k.a. entity type or entity set – a set of entities with the same characteristics e.g. patients of a hospital
- As organisations make decisions based on entities, they should be:
  - identified,
  - their attributes should be described,
  - their relationships have to be identified e.g. one order may refer to many products.

Entity-relationship diagram
Diagram encji-relacji

- Entity-relationship diagram (ER diagram) is a graphical diagram that depicts entities, their relationships and attributes
- De facto standard in data modelling as :
  - It allows for a detailed description of a data model and database structure
  - There are software tools that allow to generate CREATE TABLE statements reflecting E-R diagram (and create a database out of a diagram) and perform reverse engineering i.e. generate E-R diagram based on a database structure (e.g. Ms SQL Server database)
- Used at all stages of a database system life cycle starting from project initiation, through planning, execution, up to maintenance
- Entity-relationship diagrams can be developed both for logical and physical data models

E-R diagram - example

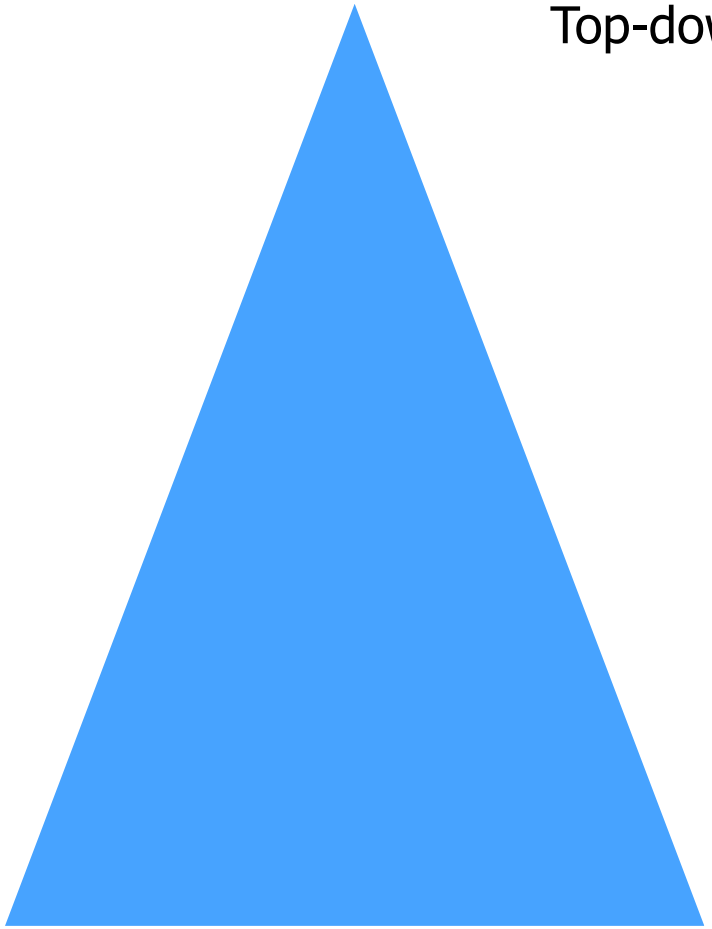


1. **Every relationship is described by a line**
2. **The line endings define the category of a relationship**

ER diagram – how to create it?

- Identify entities i.e. determine objects, people and relationships that need to be described in the system
- As there are many different notations for E-R diagrams, select one of them
- This lecture shows one of the standards
- A logical data model can be developed by developing an ER diagram

Design approaches



Top-down and bottom-up

Approach	Description
Top-down	• Start from planned benefits, scope and objectives of your project first. Identify organisation's entities first, then capture required attributes and determine relationships. • Typically used at least at the beginning of a project, since typically a project starts with e.g. a project charter defining high level project objectives.
Bottom-up	• Start from the requirements of different end users to determine different possible entities and merge them next. • Try to end up with a consistent vision of a system to be created. Less frequently used than top-down.
Used in reality	Mixture of both, first top-down allows to better capture overall requirements for the system and select its architecture, while bottom-up used next helps you answer the needs of different end users and cross-check the requirements captured with top-down.

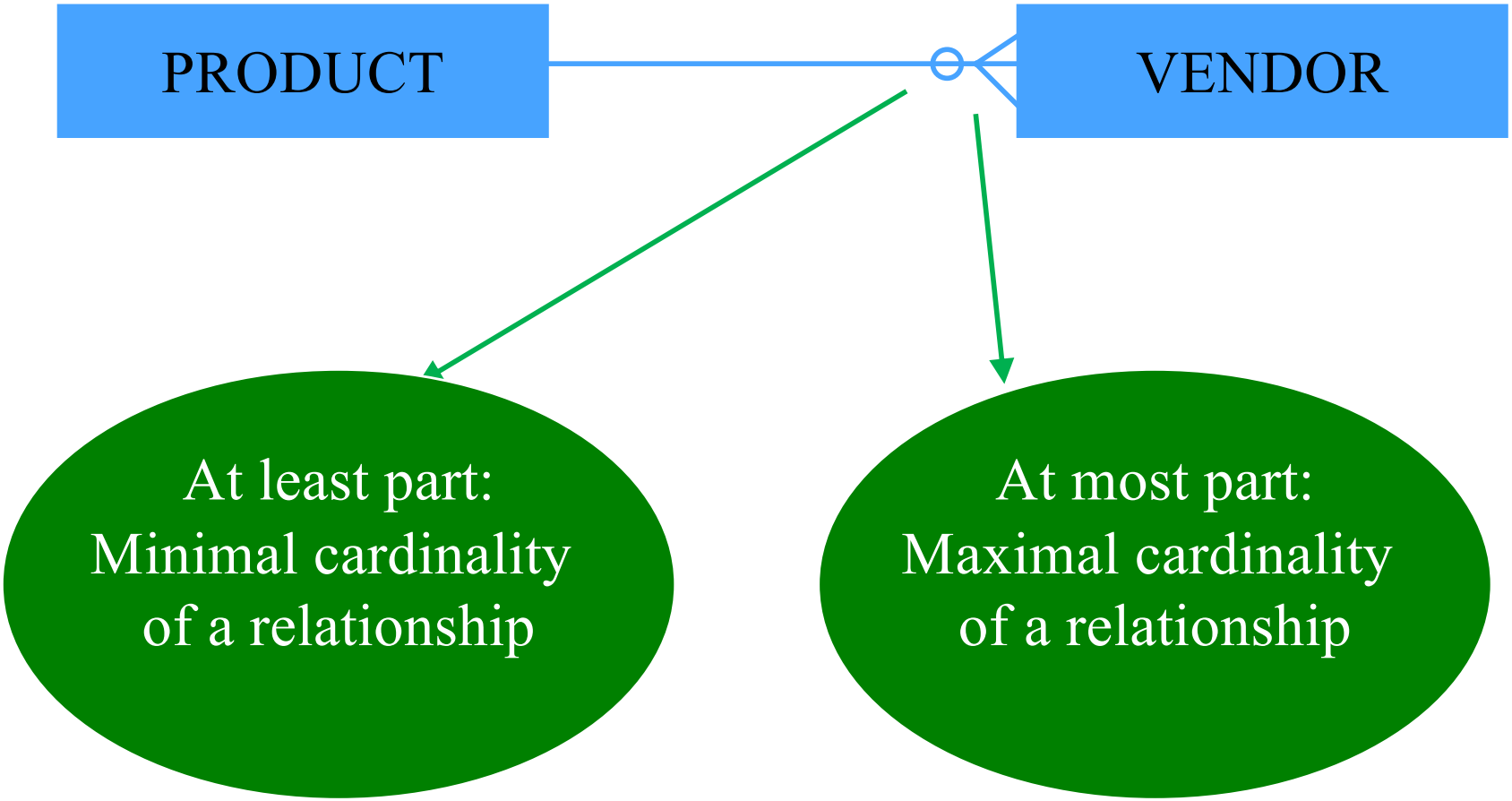
Top-down and bottom-up - guidelines

Approach	Description
Top-down	Organisation's management staff should provide business objectives for the system. Frequently economic justification for the system expressed through ROI and NPV is expected as well. It is good to start from identifying departments of an organisation to be supported by a system and entities they need.
Bottom-up	Interviews with key stakeholders of the project should provide you with their perspective on the system and its functionality. Remember to investigate the processes to be supported by the system not to be left with the users' imagination of the system. It may require a lot of experience to combine the opinions of different users into a consistent system design.
Used in reality	Usually, the management defines the scope of the project, thus you start from top-down in fact. Then you verify your initial view of the system by using bottom-up and top-down.

Design guidelines

- Reality:
  - Frequently not a very clear role of the system to be created in an organisation
  - The system is frequently expected to optimise business processes, thus a system designer has to both:
    - design a new system
    - and participate in redefining existing business processes and procedures.
- Thus:
  - Try to determine real reasons for creating a database system and problems to be solved.

Relations


PRODUCT
VENDOR
At least part:
Minimal cardinality
of a relationship
At most part:
Maximal cardinality
of a relationship

Relationship cardinality

1. Stands for zero
2. Used with minimal cardinality only

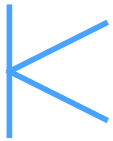
1. Stands for one
2. Used with both minimal and maximal cardinality

1. Stands for many
2. Used with maximal cardinality only

Sample relationship cardinalities

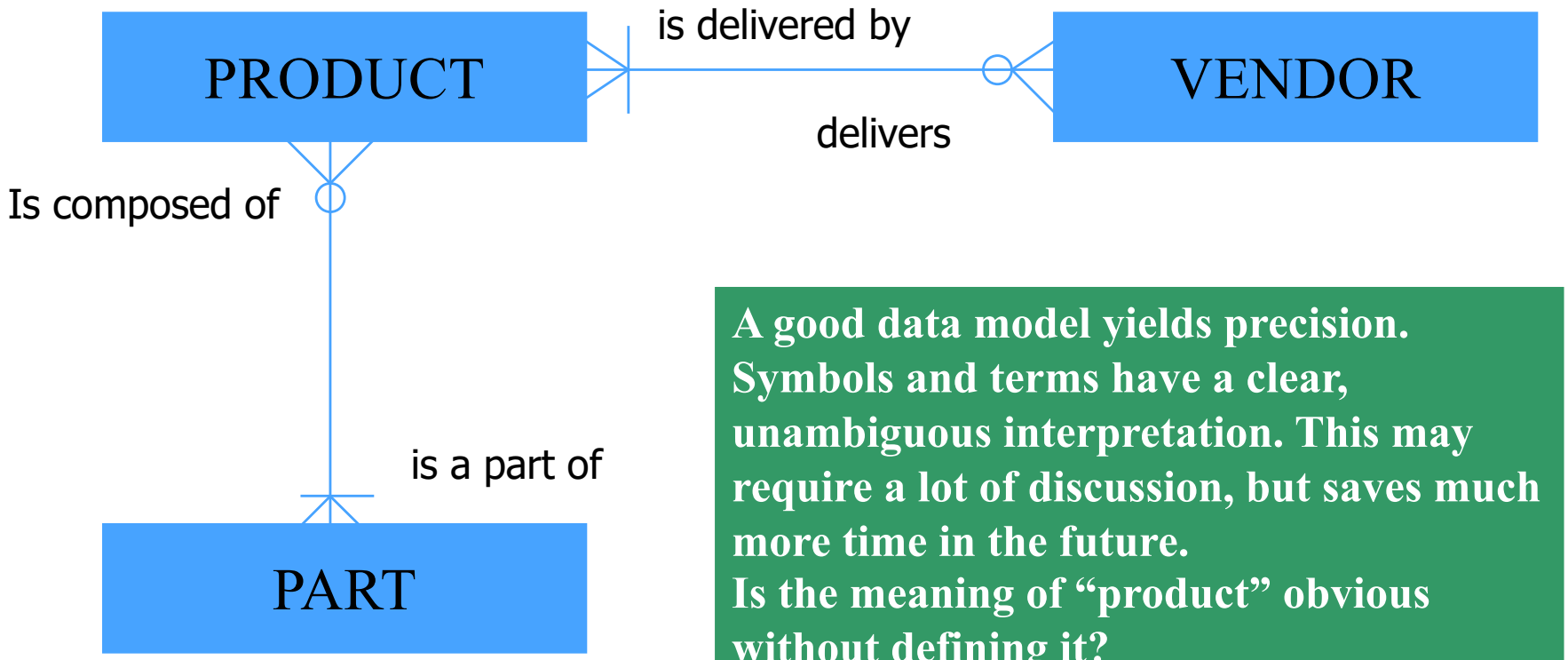
1. At least 0 at most one
2. Simply at most one e.g. every mountain is in at most one national park

1. Exactly one
2. Every capital is a capital of exactly one country



1. Stands for one to many
2. Example: every student studies on at least one faculty (but possibly many)

Sample diagram revisited: the need for precision

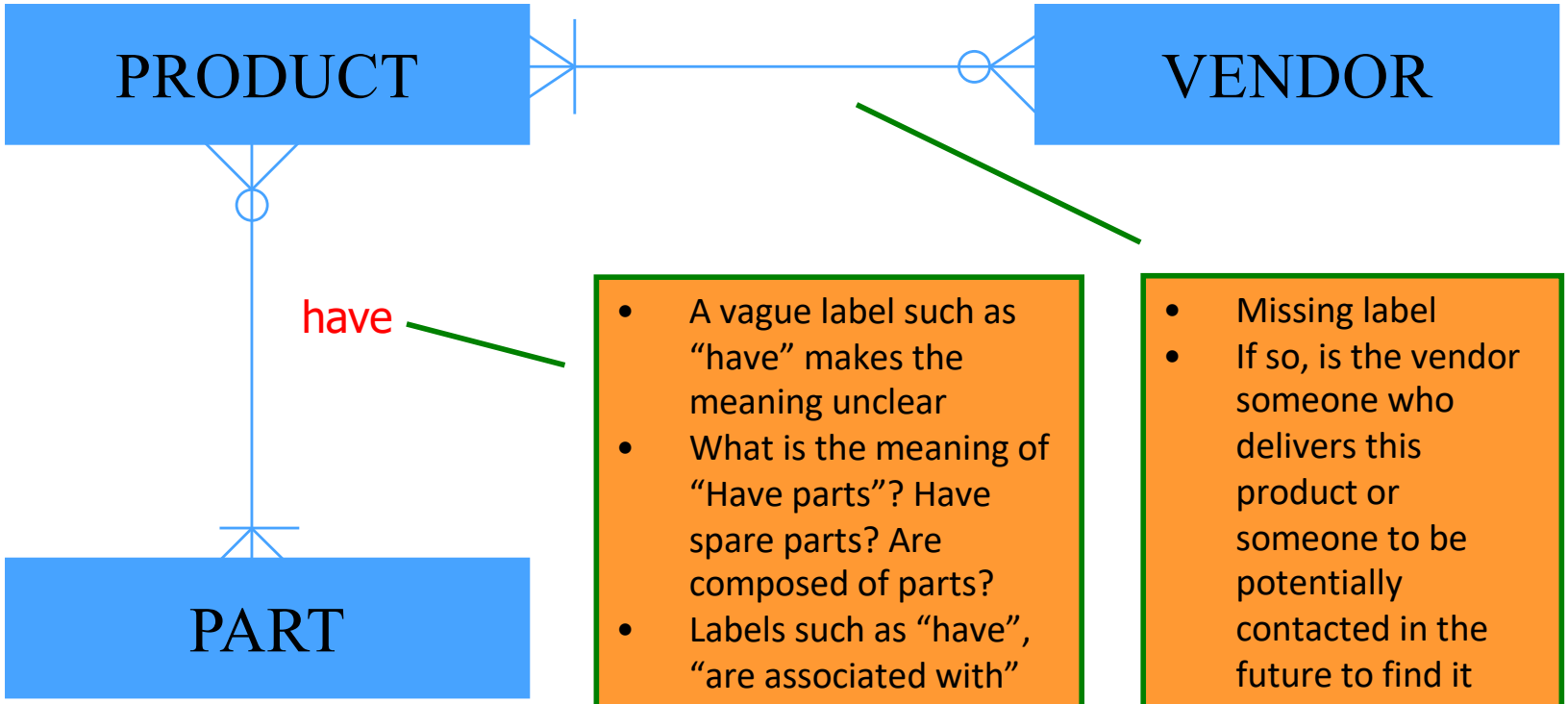


A good data model yields precision. Symbols and terms have a clear, unambiguous interpretation. This may require a lot of discussion, but saves much more time in the future.
Is the meaning of "product" obvious without defining it?

" (…) a good data model serves as a lingua franca between business and information-technology professionals. (…) Conceptual and logical data models capture the ways in which technical professionals think a business process works. Business professionals can examine these assumptions and offer corrections and refinements before code is created"

Foreword to Steve Hoberman, Data Modeling Made Simple, Technics Publications 2016

Sample diagram revisited: the risk of lost precision

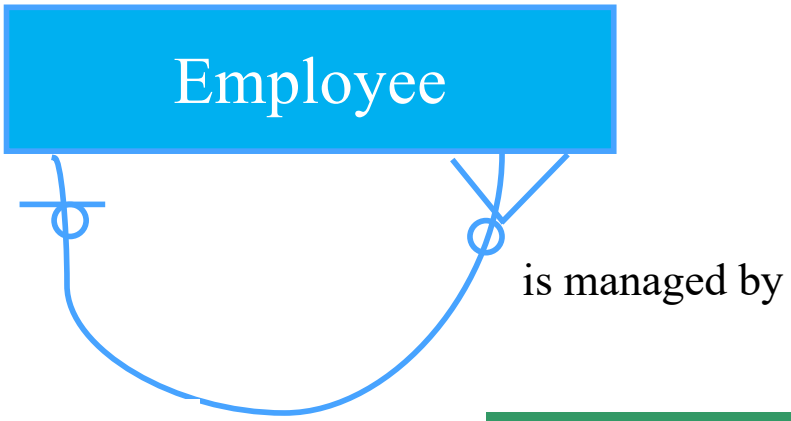


If the terms (such as product here) are poorly defined or have no definitions, the model remains imprecise

- A vague label such as “have” makes the meaning unclear
- What is the meaning of “Have parts”? Have spare parts? Are composed of parts?
- Labels such as “have”, “are associated with” etc. should be avoided

- Missing label
- If so, is the vendor someone who delivers this product or someone to be potentially contacted in the future to find it out?

Recurrent relationships



1. **It is possible that an entity enters a recurrent relationship with itself**
2. **Frequently, this corresponds to hierarchical structures of an organisation, product etc.**
3. **Note that by defining a foreign key referring to the table it is located in, we can document tree-like structure in a database**

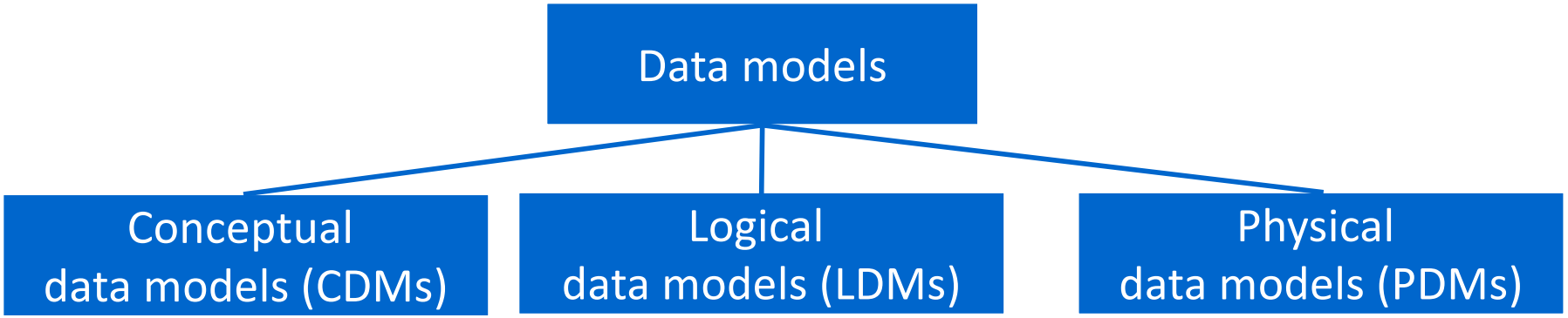
Entity description

- Entity attributes are usually written in the entity box. Frequently they correspond to column names.
- Every attribute should be assigned its domain e.g. phone numbers with (or without) country code, past dates only etc.

Name
Price
VAT rate
….

1. **Notice that you may frequently determine large number of entities and attributes in an organisation.**
2. **When your task is to develop a database application and you work on application data model, use the system scope to determine the attributes and entities you need!**
3. **When you develop an application, for every entity there must be a method for providing entity instances (input screens, data import etc.)**
4. **What would be the domain for salary attribute?**
5. **Will the domain of an attribute be important for a software developer implementing input screens or a data scientist?**

Data models revisited



- Contain only basic and critical to the business concepts, e.g. Customer, Product and Employee
- Note that some attributes can be so critical for business that can show up on CDMs, e.g. phone number in telco industry
- Conceptual relationships connect key concepts

- Entities in LDMs are more detailed than concepts in CDMs
- One concept of CDM can be represented by multiple entities in LDM
- Logical relationships are detailed business rules
- **All attributes of entities and their domains are identified**

Entities in PDMs (physical entities) are even more detailed:

- In RDBMs entities present in PDMs become tables (or views) and attributes become columns
- Physical entities contain detailed features of attributes e.g. state that CompanyName is char(50) not null
- Physical relationships become database constraints e.g. foreign keys

The objective of data modeling: sample logical data model

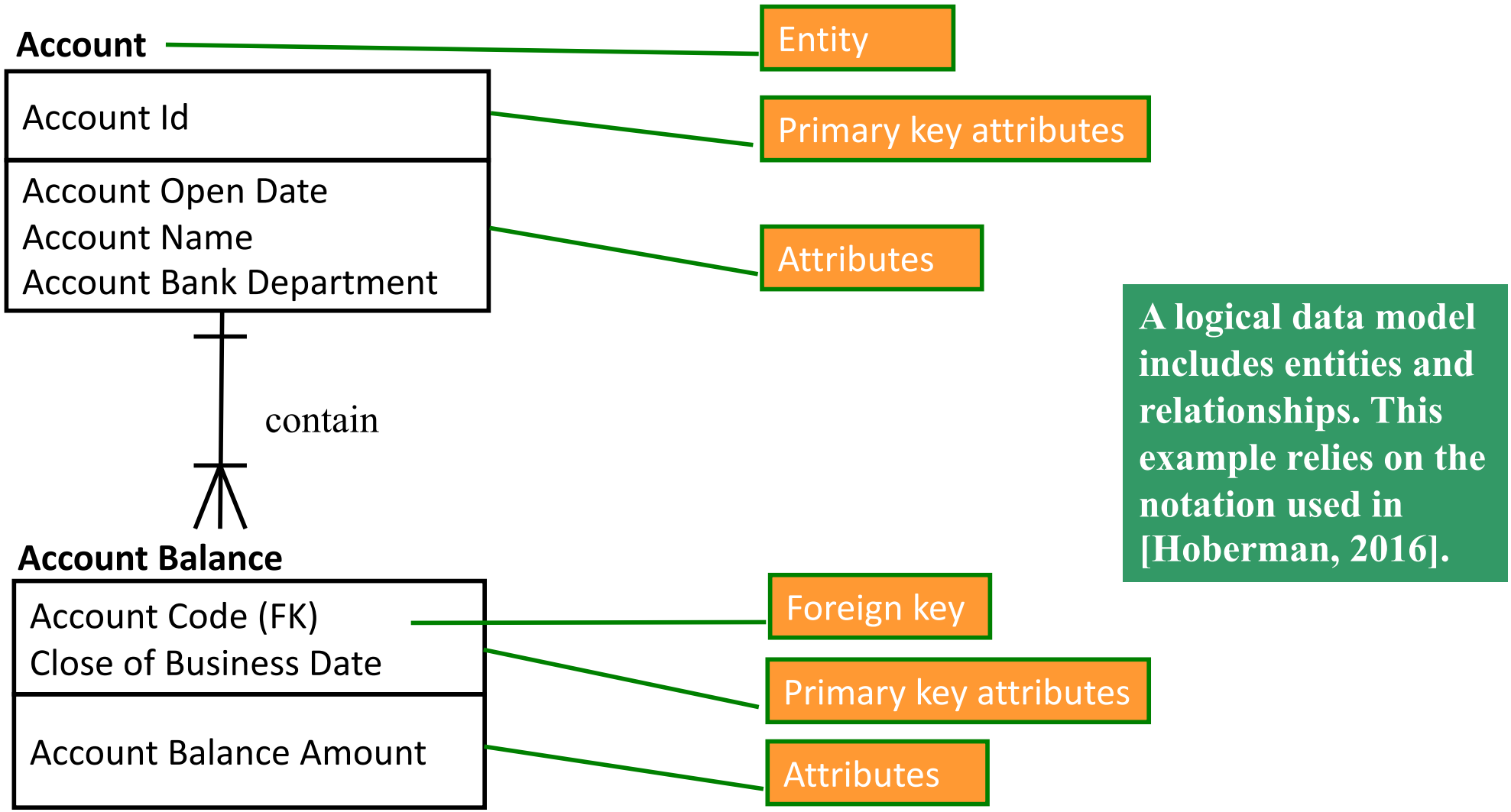
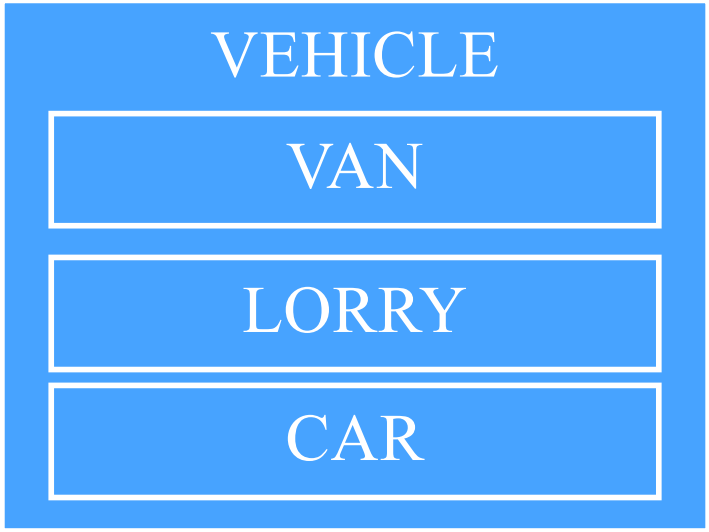


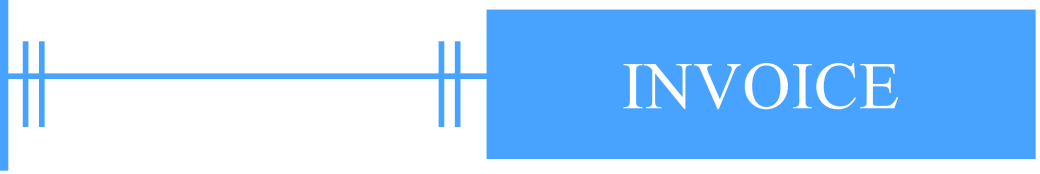
Fig. 1 Sample subset of financial LDM (based on [Hoberman, 2016])

Entities - subtypes



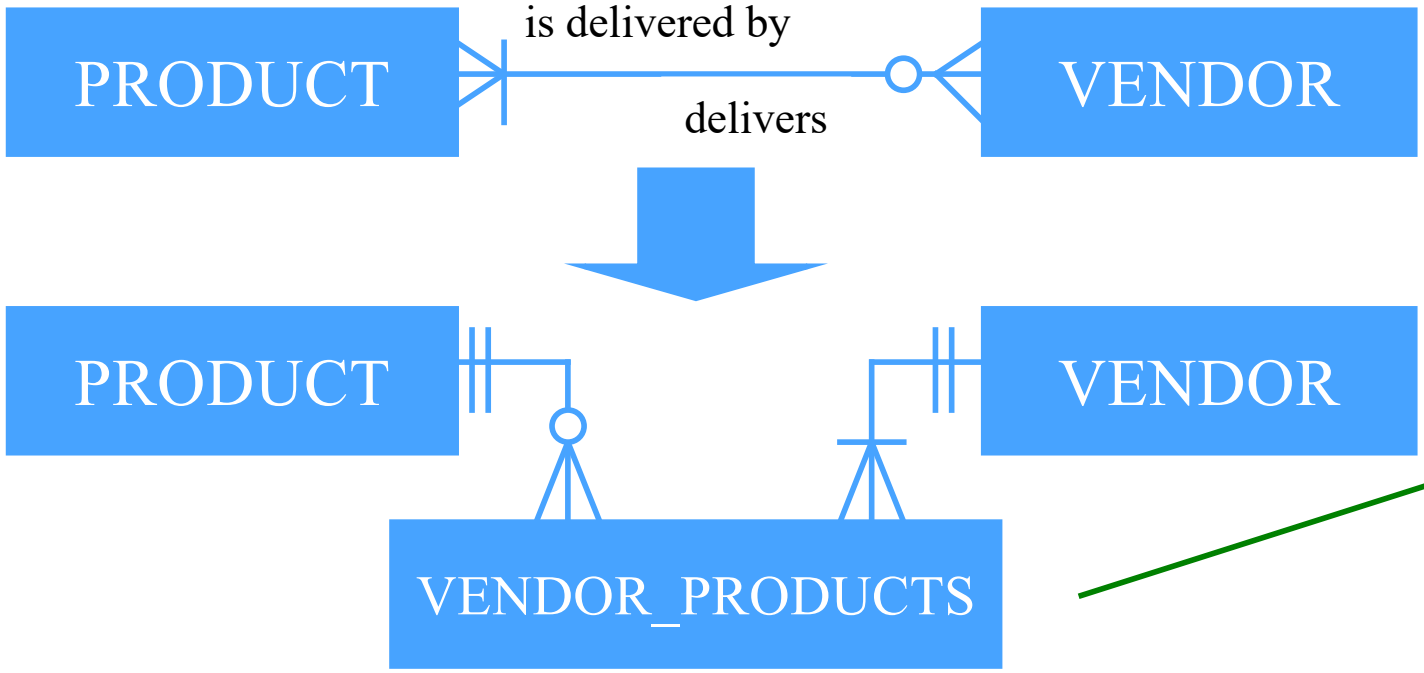
1. **In case an entity contains several different subtypes these subtypes can be depicted on the diagram as well**
2. **Remember to describe all existing subtypes in that case**

1. **Entity subtypes may also enter relationships with other entities**
2. **The more details of this kind we capture, the higher the chances for a valid model i.e. matching real life**



Many to many relationships
Relacje wiele-do-wielu

- A many-to-many relationship
  - is a relationship with “at most many” on both ends
  - is allowed in conceptual data models
  - when developing a physical model for RDBMS, should be extended with an extra entity on a diagram. Such an entity corresponds to an additional table needed in a RDBMS for the many-to-many relation.

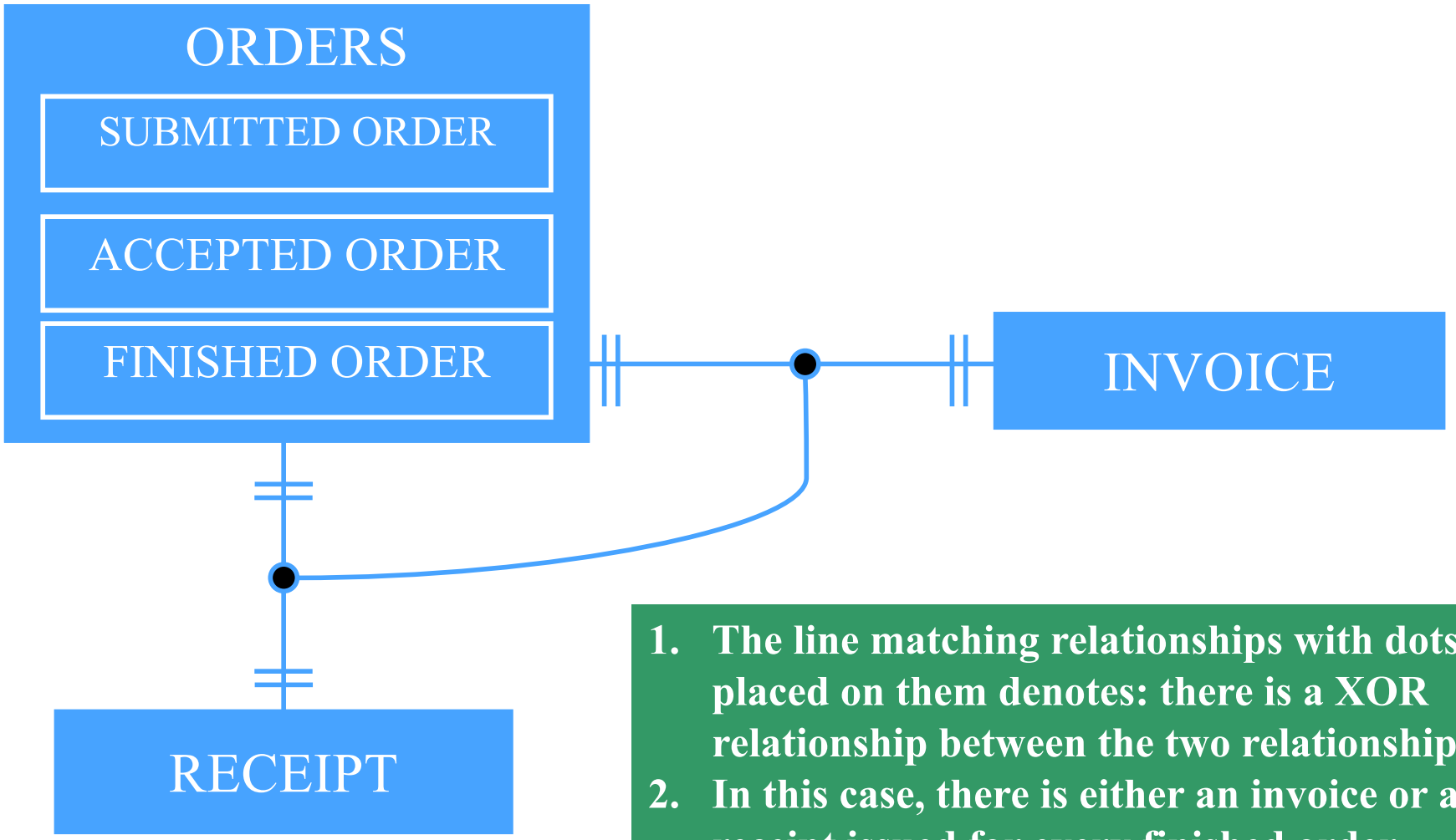


This table includes at least:
- a product identifier i.e. a FK linking it with Product table
- a vendor identifier i.e. a FK linking it with Vendor table

Mutually exclusive relationships
Relacje wzajemnie wykluczające się

- In some cases, when an entity enters a relationship with another entity, it can not participate in another relation
- Examples:
  - For every finished order, there can be either a receipt or an invoice
  - A customer is either a person or a legal entity

Mutually exclusive relationships



1. **The line matching relationships with dots placed on them denotes: there is a XOR relationship between the two relationships**
2. **In this case, there is either an invoice or a receipt issued for every finished order**

Remaining relationship properties

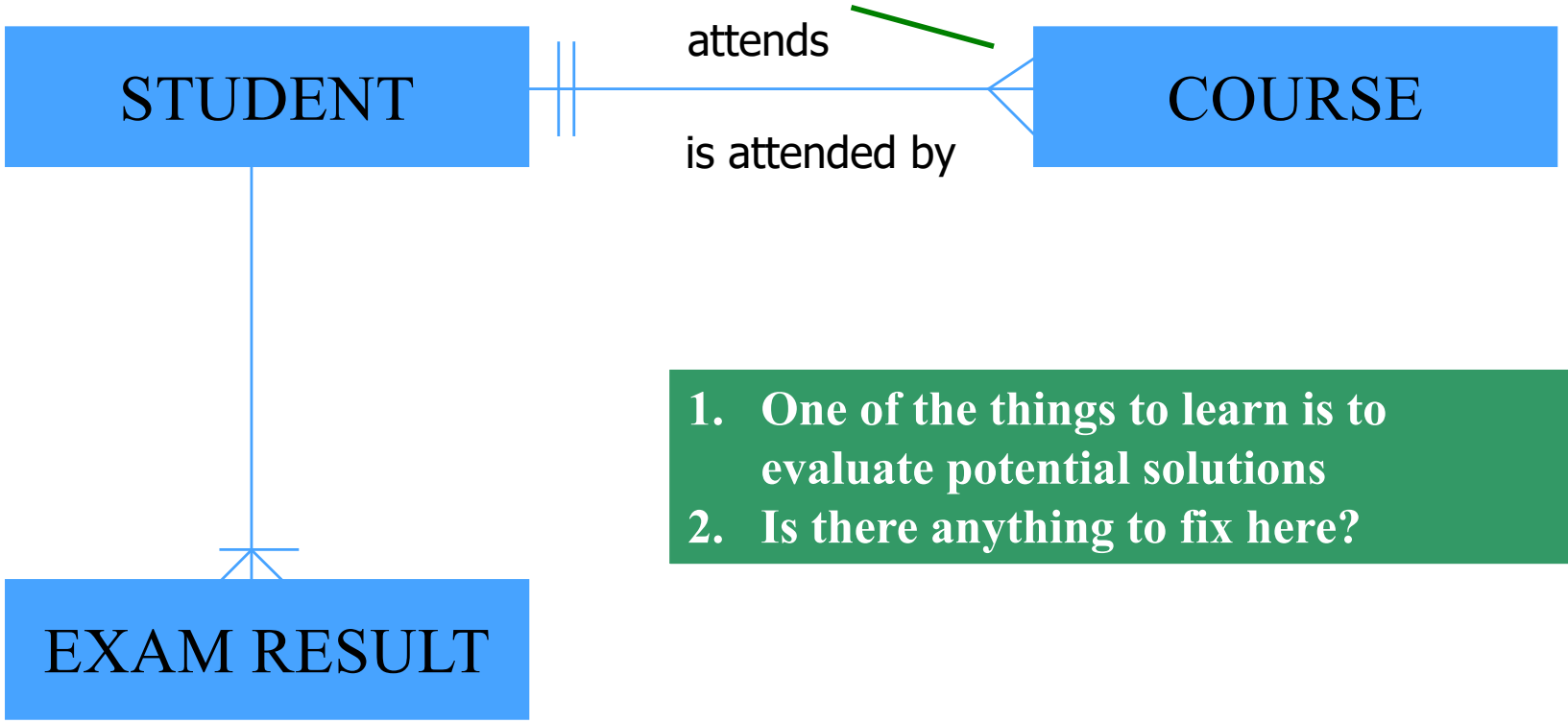
- In some CASE tools it is possible to set additional relationship properties e.g.:
  - Cardinality of relationship greater than 1 but limited e.g. each house may have at most 2 owners,
  - In case the diagram is expected to be transformed into relational database, „cascade on delete” settings can be provided for relationships e.g. for relationship between an order and order line to delete all order lines whenever an order is removed
  - relationships regarding candidate keys (i.e. unique attribute combinations) can be further described
  - Attribute i.e. column features such as declaring which columns are NOT NULL

Data modelling

- Data modelling is a vital step towards a fully functional database system
  - Logical data models help understand and document the way organisation works
  - ER diagrams help (and force team members to) capture and clearly document all relevant data needs and detailed relationships
  - The diagrams also help involve project stakeholders in the analysis process
  - When designing a database, data models are created (and modified) before the tables come into being. This reduces the risk of starting work on implementing a database and applications too early, i.e. without a proper understanding of the organisation.

What is wrong here?

Course is a component of study program. Sample courses: Programming in Python, Databases



1. **One of the things to learn is to evaluate potential solutions**
2. **Is there anything to fix here?**

Pytania ?

Zapraszam w trakcie wykładu
i w trakcie konsultacji

Miejsce i termin konsultacji
- na stronie:
https://pages.mini.pw.edu.pl/~grzendam/pl/dydaktyka.html

Questions?

I am looking forward to your questions during the lectures (including now) and during my office hours

For details on my office hours, please check my website:

https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka_eng.html

Warsaw University of Technology

MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.